Household Energy Consumption Prediction System

Comprehensive Technical Notes (Simple & Neat)

---

1. Project Overview

An intelligent system that predicts household energy consumption using machine learning.

Main Components

1. **energy_prediction_models.py**
   → Model training and evaluation
2. **app.py**
   → Flask web app for real-time predictions
3. **visualize_results.py**
   → Charts and performance analysis

Purpose

Predict total household energy usage using:

- Weather conditions
- Appliance usage
- Time patterns

---

2. Theoretical Foundation

2.1 Problem Formulation

Supervised Regression:

$$f : X \rightarrow Y$$

- **X** = Input features (weather, time, appliances)
- **Y** = Energy consumption (kWh)
- **f** = Learned prediction function

2.2 Feature Engineering

A. Temporal Features

- Hour (0–23) → Daily pattern
- Day of Week (0–6) → Weekday vs Weekend
- Month (1–12) → Seasonal trend
- Quarter (1–4) → Long-term pattern
- Week of Year

B. Weather Features

- Temperature (°C) → HVAC usage driver
- Humidity (%) → Cooling efficiency
- Wind Speed (m/s) → Natural cooling
- Solar Radiation (W/m²) → Heat gain

C. Appliance Features

- Individual appliance energy use
- Lag features (previous day energy)

---

3. Machine Learning Models

3.1 Linear Regression (Baseline Model)

Formula

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \ldots + \beta_n x_n$$

- β_0 = Intercept
- β_i = Feature coefficients
- x_i = Feature values

Error Minimization

$$MSE = (1/n)\ \Sigma(y_i - \hat{y}_i)^2$$

Role

- Baseline comparison
- Interpretable
- Fast

3.2 Random Forest Regressor (Best Model)

Ensemble of multiple decision trees.

Formula

$$\hat{y} = (1/B) \Sigma T_b(x)$$

- B = Number of trees (100)
- T_b = Each decision tree

Concept
Final prediction = Average of all trees.

Advantages

- Handles non-linear patterns
- Robust to outliers
- Feature importance ranking

3.3 Gradient Boosting Regressor

Builds models sequentially to correct errors.

Formula

$$F_m(x) = F_{m-1}(x) + \gamma_m h_m(x)$$

- F_m = Current model
- γ_m = Learning rate
- h_m = Weak learner

Residual

$$r = Actual - Predicted$$

3.4 LSTM Neural Network

Used for time-series prediction.

Core Idea
Keeps memory of past data.

Key Equations

Forget Gate

$$f_t = \sigma(W_f [h_{t-1}, x_t] + b_f)$$

Input Gate

$$i_t = \sigma(W_i [h_{t-1}, x_t] + b_i)$$

Memory Update

$$C_t = f_t \cdot C_{t-1} + i_t \cdot \tilde{C_t}$$

Output

$$h_t = o_t \cdot \tanh(C_t)$$

Architecture

- Input Layer
- LSTM Layer (50 neurons)
- Dropout (20%)
- LSTM Layer (50 neurons)
- Dense Layer (25 neurons, ReLU)
- Output Layer (1 neuron)

---

4. Evaluation Metrics

4.1 Mean Absolute Error (MAE)

$$MAE = (1/n) \Sigma |Actual - Predicted|$$

- Average absolute error
- Less sensitive to outliers

4.2 Root Mean Square Error (RMSE)

$$RMSE = \sqrt{[(1/n) \Sigma (Actual - Predicted)^2]}$$

- Penalizes large errors
- Sensitive to outliers

4.3 R-Squared (R^2)

$$R^2 = 1 - (SS_{res} / SS_{tot})$$

- 1 → Perfect
- 0 → Same as mean
- <0 → Worse than mean

---

5. Data Preprocessing

5.1 Normalization (MinMax Scaling)

```
Scaled = (X − Min) / (Max − Min)
```

Range: 0 to 1

Inverse Scaling

```
Original = (Scaled × (Max − Min)) + Min
```

5.2 Missing Values

```
Fill missing = Median of column
```

- Robust to outliers
- Preserves distribution

---

6. Web Application Architecture

6.1 Technology Stack

Backend → Flask
Frontend → HTML, CSS, JavaScript
Charts → Chart.js
API → REST endpoints

6.2 Prediction Pipeline

1. User inputs data
2. Feature engineering
3. Normalization
4. Model selection
5. Prediction
6. Post-processing

6.3 Fallback Prediction (When Models Fail)

```
Predicted Energy =
Base Energy × Temperature Factor × Humidity Factor × Solar Factor
```

Temperature Factor:

- 30°C → 1.3
- 25°C → 1.15
- <15°C → 1.2
- Else → 1.0

Humidity Factor:

```
1 + max(0, Humidity − 60) / 1000
```

Solar Factor:

```
1 + SolarRadiation / 10000
```

6.4 Energy Efficiency Logic

Waste Percentage:

```
((Predicted − ApplianceEnergy) / ApplianceEnergy) × 100
```

Efficiency:

- ≤10% → Efficient
- ≤25% → Moderate
- 25% → Wasteful

6.5 Cost Savings

```
Daily Savings = Energy Saved × Cost per kWh
Monthly = Daily × 30
Yearly = Daily × 365
```

---

7. Energy & Appliance Calculations

Appliance Energy:

```
Energy (kWh) = (Watts × Hours) / 1000
```

---

8. Time Feature Extraction

From timestamp:

- Hour (0–23)
- Day (1–31)
- Month (1–12)
- DayOfWeek (0–6)
- Quarter:

```
(Month − 1) / 3 + 1
```

- Week of Year

---

9. Neural Network Core Formula

Basic NN:

```
Output = Activation(Weights × Inputs + Bias)
```

---

10. Season Detection

```
If Temp > 30 → Hot
If Temp ≥ 20 → Moderate
Else → Cold
```

---

11. Visualization System

Charts Used:

- Bar → MAE, RMSE comparison
- Scatter → Actual vs Predicted
- Residual → Error spread
- Time Series → Trend accuracy

Statistical Tools:

- Model ranking (R^2)
- Error distribution
- Feature importance

---

12. Technologies Used

ML & Data

- scikit-learn
- TensorFlow / Keras

- pandas
- numpy
- matplotlib / seaborn

Web

- Flask
- HTML / CSS
- JavaScript
- Chart.js

Utilities

- joblib
- MinMaxScaler
- train_test_split

---

13. System Workflow

Training Phase

1. Load data
2. Clean & preprocess
3. Feature engineering
4. Train models
5. Evaluate
6. Select best
7. Save model

Prediction Phase

1. Collect inputs
2. Feature engineering
3. Normalize
4. Predict
5. Apply rules
6. Show analysis

---

14. Performance Results

- Random Forest → $R^2 = 0.9184$ (Best)
- Gradient Boosting → $R^2 = 0.8500$
- Linear Regression → $R^2 = 0.6190$
- LSTM → Varies with data

Key Insights

- Ensemble models perform best
- Non-linear models capture complex patterns
- Feature engineering improves accuracy

---

15. Conclusion

The system:

- Combines multiple ML models
- Provides real-time predictions
- Includes visualization tools
- Gives energy efficiency insights
- Achieves high accuracy ($R^2 > 0.91$)

It is a practical ML application for household energy optimization.